

Practical : 1	Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort
Date : 12/08/2026	

Objective :- To implement Bubble Sort, Selection Sort, Insertion Sort, Merge Sort and Quick Sort in C++, and to analyse and compare their time complexity (best, average, worst case) through practical execution-time measurements on inputs of varying size.

Program :-

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>
using namespace std;
using namespace std::chrono;

// ----- Bubble Sort -----
void bubbleSort(vector<int> arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
}

// ----- Selection Sort -----
void selectionSort(vector<int> arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIdx])
                minIdx = j;
        swap(arr[i], arr[minIdx]);
    }
}

// ----- Insertion Sort -----
void insertionSort(vector<int> arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

```
// ----- Merge Sort -----
void merge(vector<int> &arr, int l, int m, int r) {
    vector<int> left(arr.begin() + l, arr.begin() + m + 1);
    vector<int> right(arr.begin() + m + 1, arr.begin() + r + 1);
    int i = 0, j = 0, k = l;
    while (i < (int)left.size() && j < (int)right.size())
        arr[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];
    while (i < (int)left.size()) arr[k++] = left[i++];
    while (j < (int)right.size()) arr[k++] = right[j++];
}

void mergeSortHelper(vector<int> &arr, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSortHelper(arr, l, m);
    mergeSortHelper(arr, m + 1, r);
    merge(arr, l, m, r);
}

void mergeSort(vector<int> arr) {
    mergeSortHelper(arr, 0, arr.size() - 1);
}

// ----- Quick Sort -----
int partition(vector<int> &arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot)
            swap(arr[++i], arr[j]);
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSortHelper(vector<int> &arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSortHelper(arr, low, pi - 1);
        quickSortHelper(arr, pi + 1, high);
    }
}

void quickSort(vector<int> arr) {
    quickSortHelper(arr, 0, arr.size() - 1);
}

// ----- Timing helper -----
template <typename Func>
double measureTime(Func f, vector<int> &arr) {
    auto start = high_resolution_clock::now();
    f(arr);
    auto stop = high_resolution_clock::now();
    return duration_cast<microseconds>(stop - start).count();
}
```

```
int main() {
    srand((unsigned)time(0));
    vector<int> sizes = {100, 1000, 5000, 10000};

    cout << "Size\tBubble\tSelection\tInsertion\tMerge\tQuick\t(all times in
microseconds)\n";

    for (int n : sizes) {
        vector<int> arr(n);
        for (int i = 0; i < n; i++)
            arr[i] = rand() % 100000;

        double tBubble = measureTime(bubbleSort, arr);
        double tSelection = measureTime(selectionSort, arr);
        double tInsertion = measureTime(insertionSort, arr);
        double tMerge = measureTime(mergeSort, arr);
        double tQuick = measureTime(quickSort, arr);

        cout << n << "\t" << tBubble << "\t" << tSelection << "\t\t"
            << tInsertion << "\t\t" << tMerge << "\t" << tQuick << "\n";
    }
    return 0;
}
```

Analysis:-

Time Complexity Comparison:

Algorithm	Best Case	Average Case	Worst Case	Space	Stable
Bubble Sort	$\Theta(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n^2)$	$O(\log n)$	No

Observed Execution Time (in microseconds) on a randomly generated array:

Input Size (n)	Bubble	Selection	Insertion	Merge	Quick
100	29	10	3	12	4
1000	1884	446	100	86	42
5000	45729	8108	2367	519	259
10000	198080	31999	9644	1032	570

Bubble Sort and Selection Sort show clear quadratic growth as n increases, with Bubble Sort performing worst due to repeated adjacent swaps. Insertion Sort performs better than both on this nearly-random data but still degrades quadratically for larger n . Merge Sort and Quick Sort scale far better, closely following the $n \log n$ trend; Quick Sort is fastest in practice here due to lower constant overhead, though its worst case remains $O(n^2)$ for already sorted or adversarial inputs, which Merge Sort avoids by guaranteeing $O(n \log n)$ in every case at the cost of extra $O(n)$ space.

Conclusion :- Bubble Sort, Selection Sort and Insertion Sort were successfully implemented and shown to be simple but inefficient for large datasets, all running in $O(n^2)$ time in the average and worst cases. Merge Sort and Quick Sort were implemented as efficient $O(n \log n)$ divide-and-conquer algorithms, with practical execution times confirming their superior scalability. Quick Sort was fastest overall on random data, while Merge Sort offered more predictable, stable performance regardless of input arrangement. This confirms that algorithm choice should depend on input size, memory constraints, and whether stability or worst-case guarantees matter.